

06. 자료 구조와 알고리즘

11. 힙

학습목표

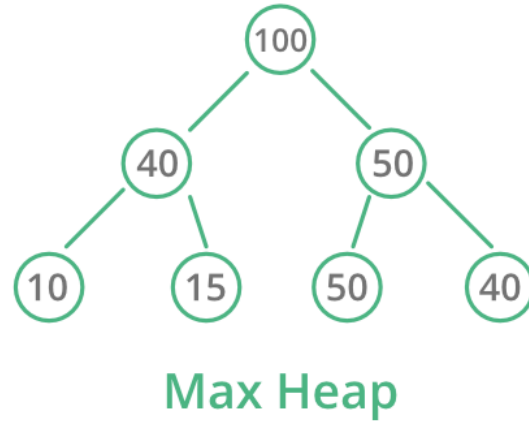
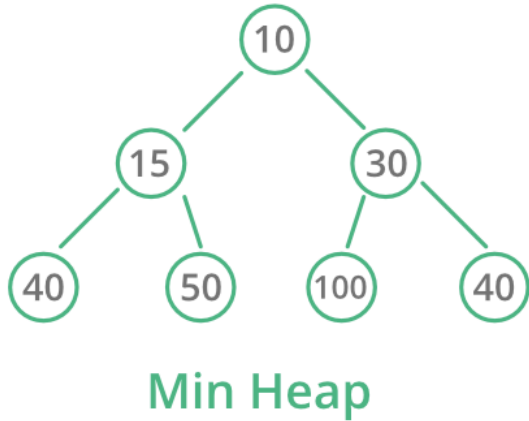
- 힙이 무엇인지 알 수 있다.
- 힙을 적재적소에 활용할 수 있다.

들어가며

힙(Heap)은 **완전 이진 트리***에 있는 노드 중에서 **값이 가장 큰 노드나 값이 가장 작은 노드**를 찾기 위해 만든 자료 구조다. 값이 가장 큰 노드를 찾기 위한 힙을 **최대 힙**(Max Heap), 가장 작은 노드를 찾기 위한 힙을 **최소 힙**(Min Heap)이라고 한다. 힙은 **우선순위 큐**(Priority Queue)라고도 한다. 이번 시간에는 힙에 대해서 알아보도록 하자.

* 완전 이진 트리(Complete Binary Tree) : 마지막 레벨을 제외하고 모든 노드의 차수가 2인 이진 트리

Heap Data Structure



GG

힉의 모습

힉의 불변성

힉이 되기 위한 조건이 있다.

- 1) 최대 / 최소 원소에 **즉각적으로 접근이 가능**해야 한다.
그래서 최대 / 최소 원소는 항상 루트 노드에 존재한다.
- 2) 부모 노드가 자식 노드보다 **항상 크거나**(Max Heap), **작아야 한다**(Min Heap).

성능

- 검색 및 읽기
 - 최대 / 최소 원소에 대해서만 가능하며 $O(1)$ 이다.
- 삽입
 - 완전 이진 트리이기 때문에 $O(\log N)$ 이다.
- 삭제
 - 최대 / 최소 원소에 대해서만 가능하며 $O(\log N)$ 이다.

사용법

힙의 경우 [PriorityQueue](#)로 제공된다.

```
// 기본은 최소 힙이다.

// 첫 번째 타입 매개변수는 요소(Element)를 두 번째 타입 매개변수는 우선순위(Priority)를 나타낸다.
PriorityQueue<string, int> minHeap = new();

// # 삽입

// Enqueue()로 삽입을 할 수 있다.
minHeap.Enqueue("강호동", 3);
minHeap.Enqueue("유재석", 1);
minHeap.Enqueue("신동엽", 2);

// # 접근

// Peek()로 루트 노드에 접근할 수 있다.
Console.WriteLine(minHeap.Peek()); // "유재석"

// # 삭제

// Dequeue()로 루트 노드를 삭제할 수 있다.
Console.WriteLine(minHeap.Dequeue()); // "유재석"
Console.WriteLine(minHeap.Dequeue()); // "신동엽"
Console.WriteLine(minHeap.Dequeue()); // "강호동"

// 최대 힙을 만들고 싶다면 IComparer를 구현하면 된다.
class Greater : IComparer<int>
{
```

```

public int Compare(int x, int y)
{
    return y - x;
}
}

PriorityQueue<string, int> maxHeap = new PriorityQueue<string, int>(new Greater());

minHeap.Enqueue("강호동", 3);
minHeap.Enqueue("유재석", 1);
minHeap.Enqueue("신동엽", 2);

Console.WriteLine(minHeap.Peek()); // "강호동"

```

구현

하지만, PriorityQueue의 경우 최신 라이브러리나 프로젝트에 따라선 사용이 불가능할 수 있다. 따라서 직접 구현해보도록 하자.

```

class MinHeap
{
    // 완전 이진 트리가기 때문에 배열로 구현할 수 있다.
    // 낭비되는 메모리가 없기 때문이다.

    private List<int> _container = new List<int>();

    public int Peek()

```

```

{
    // 루트 노드는 0번 인덱스에 존재한다.
    return _container[0];
}

public void Enqueue(int value)
{
    // 1. 맨 마지막에 삽입한다.
    _container.Add(value);

    // 2. 힙의 불변성을 만족할 때까지 데이터를 부모와 바꿔준다.
    int current = _container.Count;
    while (true)
    {
        // HACK : 계산 편의성을 위해 인덱스를 1부터 시작했으므로 데이터에 접근하려면 1을 빼야 한다.

        // 루트 노드까지 탐색한 것이다.
        if (current == 1)
        {
            break;
        }

        // 힙의 불변성을 다 만족했다.
        int parent = current / 2;
        if (_container[parent - 1] < _container[current - 1])
        {

```

```

        break;

    }

    // 부모와 바꿔준다.

    int temp = _container[current - 1];

    _container[current - 1] = _container[parent - 1];

    _container[parent - 1] = temp;

    // current를 이동시킨다.

    current = parent;
}
}

public int Dequeue()
{
    // 1. 루트 노드를 제거 한다.

    int result = _container[0];

    // 2. 가장 마지막에 있는 노드를 루트로 가져온다.

    _container[0] = _container[_container.Count - 1];

    _container.RemoveAt(_container.Count - 1);

    // 2. 힙의 불변성을 만족할 때까지 데이터를 자식과 바꿔준다.

    int current = 1;

    while (true)

```

```

{

    int left = current * 2;

    int right = current * 2 + 1;


    // 완전 이진 트리이므로 왼쪽 자식 노드가 없다면 모든 노드를 탐색한 것이다.
    if (left > _container.Count)
    {
        break;
    }


    // 바꿀 자식을 선택한다.
    int child = left;

    if (right <= _container.Count && _container[right - 1] < _container[left - 1])
    {
        child = right;
    }


    // 힙의 불변성을 만족했다면 종료한다.
    if (_container[current - 1] < _container[child - 1])
    {
        break;
    }


    // 자식과 바꾼다.
    int temp = _container[current - 1];
    _container[current - 1] = _container[child - 1];
    _container[child - 1] = temp;
}

```

```

// current를 이동시킨다.

current = child;

}

return result;

}
}

```

더 나아가기

- 오늘 수업 내용을 복기해보며 아래 내용을 정리해봅시다.
 - 힙이란 무엇인가요?
 - 힙의 불변성은 무엇인가요?
 - 힙의 성능은 어떻게 되나요?
 - 힙을 구현할 때 연결 리스트보다 선형 리스트가 선호되는 이유는 무엇인가요?
 - MinHeap을 [PriorityQueue](#)와 동일하게 제네릭 버전으로 바꿔보세요. 즉, 여러 타입으로 사용할 수 있고 비교 함수를 외부로부터 받게 수정해보세요.
- [PriorityQueue](#)의 메소드 및 프로퍼티를 정리해보세요.
- 힙을 이용하면 힙 정렬을 구현할 수 있습니다. 힙 정렬의 성능에 대해서 생각해봅시다.
- 아래 문제를 풀어보며 힙의 활용을 생각해봅시다.
 - [11279번: 최대 힙](#)
 - [1927번: 최소 힙](#)
 - [11286번: 절댓값 힙](#)

참고자료

- [C로 배우는 쉬운 자료구조](#)
- [누구나 자료 구조와 알고리즘](#)
- [PriorityQueue<TElement,TPriority> Class \(System.Collections.Generic\) | Microsoft Learn](#)

- [Binary Heap \(Priority Queue\) - VisuAlgo](#)